FIAP — Compliance, Quality Assurance & Tests — Desafio de Testes Automatizados

Testes Automatizados BDD
HealthTrack API — Cidades ESG Inteligentes

Cenários Gherkin, testes de API (status code, corpo JSON e contrato JSON Schema), passo a passo de configuração e evidências de execução

Natalia Santana Silva — RM557347

Repositório: **github.com/nataliassilva24/cidades-esg-inteligentes** — pasta `testes-bdd/`

32	199	14/14	12	100%
cenários	passos	endpoints testados	contratos JSON Schema	aprovados

Sumário

1. Objetivo e contexto ESG
2. Arquitetura da solução de testes
3. Passo a passo de configuração (com prints)
4. Cenários de teste em Gherkin
5. Testes de API: status code, corpo JSON e contrato
6. Como executar
7. Execução detalhada e evidências
8. Conclusão

1. Objetivo e contexto ESG

A HealthTrack é uma API de prontuário eletrônico unificado. Ao centralizar o histórico clínico e de exames do paciente, evita a **repetição desnecessária de exames** no sistema de saúde — impacto medido pelo indicador ESG `/indicadores/desperdicio`.

Esta entrega implementa testes automatizados com **BDD e Gherkin** para as funcionalidades críticas, em cenários positivos (caminhos felizes) e negativos (falhas), e **testes de API para todos os endpoints**, validando status code, corpo JSON e contrato via JSON Schema.

> **Relação com os pilares ESG**
> • **Social** (`@esg-social`): prontuário unificado, histórico de exames e o indicador de exames repetidos evitados — inclusive a **regra de cálculo** do indicador é testada.
> • **Governança** (`@esg-governanca`): controle de acesso por perfil (401/403), integridade dos dados (CPF e e-mail únicos), proteção de dados sensíveis de saúde (LGPD) e rastreabilidade do ambiente/versão implantados.

2. Arquitetura da solução de testes

Ferramenta	Uso
Cucumber 7 (JUnit Platform)	Executa os cenários escritos em Gherkin, em português (`# language: pt`)
RestAssured 5	Chamadas HTTP à API real; validação de status code e corpo JSON
RestAssured JSON Schema Validator	Testes de contrato: cada resposta é validada contra um JSON Schema
PicoContainer	Compartilha o contexto do cenário (token JWT, variáveis, última resposta) entre os passos
Docker / Docker Compose	Sobe API + MySQL e, opcionalmente, o container que executa os testes
GitHub Actions	Pipeline de CI que executa a suíte a cada push / Pull Request

Os testes são **caixa-preta**: chamam a API real em execução (MySQL + migrações Flyway), como um cliente faria. A massa de dados é gerada a cada execução (e-mails únicos e **CPFs válidos gerados com dígitos verificadores**), então a suíte pode ser repetida no mesmo banco sem conflitos. Os usuários de cada perfil (ADMIN, MEDICO, PACIENTE) são registrados pela própria API no início da execução.

Estrutura do projeto de testes (testes-bdd/)

```
$ cd testes-bdd && find . -type f -not -path "./target/*" -not -path "./docs/*" | sort
./Dockerfile
./README.md
./docker-compose.testes.yml
./pom.xml
./run-tests.sh
./src/test/java/br/com/fiap/healthtrack/bdd/RunCucumberTest.java
./src/test/java/br/com/fiap/healthtrack/bdd/steps/ApiSteps.java
./src/test/java/br/com/fiap/healthtrack/bdd/support/ApiContext.java
./src/test/resources/features/01_info_saude.feature
./src/test/resources/features/02_autenticacao.feature
./src/test/resources/features/03_pacientes.feature
./src/test/resources/features/04_prontuarios.feature
./src/test/resources/features/05_exames.feature
./src/test/resources/features/06_indicadores_esg.feature
./src/test/resources/schemas/erro-validacao.json
./src/test/resources/schemas/erro.json
./src/test/resources/schemas/exame.json
./src/test/resources/schemas/health.json
./src/test/resources/schemas/indicador-desperdicio.json
./src/test/resources/schemas/info.json
./src/test/resources/schemas/paciente.json
./src/test/resources/schemas/pagina-exames.json
./src/test/resources/schemas/pagina-pacientes.json
./src/test/resources/schemas/pagina-prontuarios.json
./src/test/resources/schemas/prontuario.json
./src/test/resources/schemas/token.json
```

Figura 1 — Estrutura do projeto de testes: runner, passos, contexto, 6 features e 12 contratos

3. Passo a passo de configuração

3.1 Pré-requisitos

- **Docker + Docker Compose** — obrigatório (sobe API e MySQL; também pode executar os testes).
- **Java 21+ e Maven 3.9+** — somente para executar os testes fora de container.
- **Nenhum segredo real é necessário**: localmente usa-se o `.env.example`; no CI, o workflow gera senhas e chave JWT aleatórias a cada execução. Por isso não há *secrets* nem *environments* a cadastrar no GitHub para esta suíte.

3.2 Variáveis de ambiente

O arquivo `.env` (não versionado) é criado a partir do modelo. O script de execução faz isso automaticamente se ele não existir.

Configuração do ambiente — variáveis (.env)

```
$ cp .env.example .env      # cria o .env local (valores de exemplo, sem segredos reais)
$ cat .env.example
# Copie este arquivo para .env e ajuste os valores:  cp .env.example .env
# O arquivo .env NÃO deve ser versionado (está no .gitignore).

# --- Banco de dados (MySQL) ---
MYSQL_ROOT_PASSWORD=troque-esta-senha-root
MYSQL_DATABASE=healthtrack
MYSQL_USER=healthtrack_app
MYSQL_PASSWORD=troque-esta-senha

# --- Segurança (assinatura dos tokens JWT; mínimo 32 caracteres) ---
JWT_SECRET=troque-por-uma-chave-aleatoria-com-pelo-menos-32-caracteres

# --- Aplicação ---
APP_ENV=local
APP_VERSION=dev
APP_PORT=8080
```

Figura 2 — Criação do .env a partir do .env.example (valores de exemplo, sem segredos reais)

3.3 Dependências do projeto de testes

testes-bdd/pom.xml — dependências (Cucumber, RestAssured, JSON Schema Validator)

```xml
    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <cucumber.version>7.20.1</cucumber.version>
        <junit.version>5.11.3</junit.version>
        <junit.platform.version>1.11.3</junit.platform.version>
        <restassured.version>5.5.0</restassured.version>
    </properties>

    <dependencies>
        <!-- BDD: Cucumber (Gherkin) rodando sobre a JUnit Platform -->
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-java</artifactId>
            <version>${cucumber.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-junit-platform-engine</artifactId>
            <version>${cucumber.version}</version>
            <scope>test</scope>
        </dependency>
        <!-- Injeção do contexto compartilhado entre as classes de steps -->
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-picocontainer</artifactId>
            <version>${cucumber.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.junit.platform</groupId>
            <artifactId>junit-platform-suite</artifactId>
            <version>${junit.platform.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>

        <!-- Testes de API: chamadas HTTP, validação de status code e corpo JSON -->
        <dependency>
            <groupId>io.rest-assured</groupId>
            <artifactId>rest-assured</artifactId>
            <version>${restassured.version}</version>
            <scope>test</scope>
        </dependency>
        <!-- Testes de contrato: validação da resposta contra JSON Schema -->
        <dependency>
            <groupId>io.rest-assured</groupId>
            <artifactId>json-schema-validator</artifactId>
            <version>${restassured.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.hamcrest</groupId>
            <artifactId>hamcrest</artifactId>
            <version>2.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

Figura 3 — pom.xml — Cucumber, JUnit Platform, PicoContainer, RestAssured e JSON Schema Validator

3.4 Runner da suíte

Executa todos os `.feature` e gera relatórios HTML e JSON em `target/cucumber-reports/`.

RunCucumberTest.java — runner da suíte BDD

```java
package br.com.fiap.healthtrack.bdd;

import org.junit.platform.suite.api.ConfigurationParameter;
import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;

import static io.cucumber.junit.platform.engine.Constants.GLUE_PROPERTY_NAME;
import static io.cucumber.junit.platform.engine.Constants.PLUGIN_PROPERTY_NAME;

/**
 * Ponto de entrada da suíte BDD: executa todos os arquivos .feature de src/test/resources/features.
 * Relatórios gerados em target/cucumber-reports (HTML e JSON).
 */
@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(key = GLUE_PROPERTY_NAME, value = "br.com.fiap.healthtrack.bdd")
@ConfigurationParameter(key = PLUGIN_PROPERTY_NAME,
        value = "pretty, summary, html:target/cucumber-reports/relatorio.html, json:target/cucumber-reports/relatorio.json")
public class RunCucumberTest {
}
```

Figura 4 — RunCucumberTest.java — configuração do Cucumber sobre a JUnit Platform

3.5 Definição dos passos (step definitions)

Os passos são genéricos e reutilizáveis (boa prática de Gherkin): o mesmo passo valida status code, contrato ou campos de qualquer endpoint.

ApiSteps.java — passos que validam status code, contrato e corpo JSON

```java
    @Então("o status da resposta deve ser {int}")
    public void oStatusDaRespostaDeveSer(int status) {
        ctx.ultimaResposta().then().statusCode(status);
    }

    @E("a resposta deve respeitar o contrato {string}")
    public void aRespostaDeveRespeitarOContrato(String schema) {
        ctx.ultimaResposta().then().assertThat().body(matchesJsonSchemaInClasspath("schemas/" + schema));
    }

    @E("o campo {string} deve ser {string}")
    public void oCampoDeveSer(String campo, String valor) {
        ctx.ultimaResposta().then().body(campo, hasToString(ctx.resolver(valor)));
    }

    @E("o campo {string} deve ser o número {int}")
    public void oCampoDeveSerONumero(String campo, int valor) {
        ctx.ultimaResposta().then().body(campo, equalTo(valor));
    }
```

Figura 5 — ApiSteps.java — validação de status code, contrato JSON Schema e campos do corpo JSON

3.6 Contratos (JSON Schema)

Cada tipo de resposta tem um contrato com campos obrigatórios, tipos, formatos (regex de CPF e data ISO), enumerações e `additionalProperties: false` — qualquer campo removido, renomeado ou adicionado na API quebra o teste.

schemas/paciente.json — contrato JSON Schema

```json
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "PacienteResponse",
  "type": "object",
  "required": ["id", "nome", "cpf", "dataNascimento"],
  "properties": {
    "id":             { "type": "integer", "minimum": 1 },
    "nome":           { "type": "string", "minLength": 2, "maxLength": 150 },
    "cpf":            { "type": "string", "pattern": "^\\d{3}\\.?\\d{3}\\.?\\d{3}-?\\d{2}$" },
    "dataNascimento": { "type": "string", "pattern": "^\\d{4}-\\d{2}-\\d{2}$" },
    "telefone":       { "type": ["string", "null"] },
    "email":          { "type": ["string", "null"] },
    "endereco":       { "type": ["string", "null"], "maxLength": 300 }
  },
  "additionalProperties": false
}
```

Figura 6 — schemas/paciente.json — contrato da resposta de paciente

3.7 Container de testes

testes-bdd/Dockerfile — imagem que executa os testes

```dockerfile
# Imagem que executa a suíte BDD (Cucumber + RestAssured) contra a API em execução.
# A URL da API é informada pela variável BASE_URL (padrão: serviço "api" do docker-compose).
FROM maven:3.9-eclipse-temurin-21

WORKDIR /opt/testes

# Baixa as dependências primeiro para aproveitar o cache de camadas
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

COPY src ./src

ENV BASE_URL=http://api:8080

# Os relatórios ficam em /opt/testes/target/cucumber-reports (montado como volume pelo compose).
# Sem "clean": a imagem já nasce sem target/ e o diretório montado não pode ser apagado.
ENTRYPOINT ["sh", "-c", "mvn -B test -DbaseUrl=$BASE_URL"]
```

Figura 7 — testes-bdd/Dockerfile — imagem Maven + JDK 21 que executa a suíte

testes-bdd/docker-compose.testes.yml — container de testes na rede da API

```yaml
# Serviço adicional que roda os testes BDD dentro da mesma rede da API.
# Uso (na raiz do projeto):
#   docker compose -f docker-compose.yml -f testes-bdd/docker-compose.testes.yml up --build \
#     --abort-on-container-exit --exit-code-from testes
services:
  testes:
    build: ./testes-bdd
    container_name: healthtrack-testes-bdd
    depends_on:
      api:
        condition: service_healthy
    environment:
      BASE_URL: http://api:8080
    volumes:
      - ./testes-bdd/target/cucumber-reports:/opt/testes/target/cucumber-reports
    networks:
      - healthtrack-net
```

Figura 8 — docker-compose.testes.yml — serviço de testes na mesma rede da API, aguardando o healthcheck

3.8 Script de execução

testes-bdd/run-tests.sh — execução em um comando

```bash
#!/usr/bin/env bash
# Sobe a API + MySQL com Docker Compose e executa a suíte BDD.
#
# Uso (a partir da raiz do projeto):
#   ./testes-bdd/run-tests.sh           -> executa os testes com Maven local (Java 21+)
#   ./testes-bdd/run-tests.sh --docker   -> executa os testes dentro de um container (só precisa de Docker)
#
# Variáveis opcionais: APP_PORT (porta local da API, padrão 8080)
set -euo pipefail

RAIZ="$(cd "$(dirname "$0")/.." && pwd)"
cd "$RAIZ"

APP_PORT="${APP_PORT:-8080}"
export APP_PORT

if [ ! -f .env ]; then
  echo ">> Arquivo .env não encontrado: criando a partir de .env.example"
  cp .env.example .env
fi

if [ "${1:-}" = "--docker" ]; then
  echo ">> Executando API + testes BDD em containers"
  docker compose -f docker-compose.yml -f testes-bdd/docker-compose.testes.yml up --build \
    --abort-on-container-exit --exit-code-from testes db api testes
  echo ">> Relatório: testes-bdd/target/cucumber-reports/relatorio.html"
  exit 0
fi

echo ">> Subindo API + MySQL (porta ${APP_PORT})"
docker compose up -d --build db api

echo ">> Aguardando a API ficar saudável..."
for i in $(seq 1 60); do
  if curl -fs "http://localhost:${APP_PORT}/actuator/health" > /dev/null; then
    echo ">> API pronta"
    break
  fi
  [ "$i" = 60 ] && { echo "!! API não respondeu a tempo"; docker compose logs api | tail -50; exit 1; }
  sleep 3
done

echo ">> Executando testes BDD"
cd testes-bdd
mvn -B clean test -DbaseUrl="http://localhost:${APP_PORT}"
echo ">> Relatório: testes-bdd/target/cucumber-reports/relatorio.html"
```

Figura 9 — run-tests.sh — cria o .env, sobe o ambiente, aguarda a API saudável e executa os testes

3.9 Pipeline de CI (GitHub Actions)

.github/workflows/bdd-tests.yml — pipeline de CI (GitHub Actions)

```yaml
# Testes automatizados BDD (Cucumber/Gherkin) e de API (status code, corpo JSON e contrato JSON Schema).
# Sobe a API real + MySQL com Docker Compose e executa a suíte de testes-bdd contra ela.
name: Testes BDD e de API

on:
  push:
    branches: ["feature/**", develop, main]
  pull_request:
    branches: [develop, main]
  workflow_dispatch:

jobs:
  bdd:
    name: Cucumber + RestAssured + JSON Schema
    runs-on: ubuntu-24.04

    steps:
      - name: Git Checkout
        uses: actions/checkout@v5

      - name: Criar .env de teste (segredos gerados só para esta execução)
        run: |
          cp .env.example .env
          sed -i "s|^MYSQL_ROOT_PASSWORD=.*|MYSQL_ROOT_PASSWORD=$(openssl rand -hex 16)|" .env
          sed -i "s|^MYSQL_PASSWORD=.*|MYSQL_PASSWORD=$(openssl rand -hex 16)|" .env
          sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$(openssl rand -hex 32)|" .env

      - name: Subir API + MySQL (Docker Compose)
        run: docker compose up -d --build db api

      - name: Aguardar API saudável
        run: |
          for i in $(seq 1 60); do
            curl -fs http://localhost:8080/actuator/health && exit 0
            sleep 3
          done
          docker compose logs api
          exit 1

      - name: Setup Java SDK 21
        uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Executar testes BDD
        working-directory: testes-bdd
        run: mvn -B clean test -DbaseUrl=http://localhost:8080

      - name: Publicar relatório Cucumber
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: relatorio-bdd
          path: |
            testes-bdd/target/cucumber-reports/
            testes-bdd/target/surefire-reports/

      - name: Derrubar ambiente
        if: always()
        run: docker compose down -v
```

Figura 10 — .github/workflows/bdd-tests.yml — CI que sobe a API, executa a suíte e publica o relatório

4. Cenários de teste em Gherkin

São 6 funcionalidades com cenários positivos (`@positivo`) e negativos (`@negativo`). Variáveis entre chaves (`{pacienteId}`, `{cpfValido}`, `{execucao}`) são preenchidas em tempo de execução com dados gerados no próprio cenário.

features/06_indicadores_esg.feature — cenários Gherkin (ESG Social)

```gherkin
# language: pt
@indicadores @esg-social
Funcionalidade: Indicador ESG de desperdício evitado
  Como gestor de saúde pública
  Quero acompanhar quantos exames repetidos foram evitados e a economia gerada
  Para medir o impacto social da plataforma

  @positivo
  Cenário: Consultar o indicador de exames repetidos evitados
    Dado que estou autenticado como "ADMIN"
    Quando envio uma requisição GET para "/indicadores/desperdicio"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "indicador-desperdicio.json"
    E o campo "impactoESG" deve conter "Social"

  @positivo
  Cenário: Exame repetido entra no cálculo do indicador
    Dado que existe um paciente cadastrado
    E que estou autenticado como "MEDICO"
    E envio uma requisição GET para "/indicadores/desperdicio"
    E guardo o campo "examesRepetidosEvitados" da resposta como "antes"
    E envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "CARDIOLOGICO", "descricao": "Eletrocardiograma", "dataRealizacao": "2026-09-01"}
      """
    E envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "CARDIOLOGICO", "descricao": "Eletrocardiograma", "dataRealizacao": "2026-09-20"}
      """
    Quando envio uma requisição GET para "/indicadores/desperdicio"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "indicador-desperdicio.json"
    E o indicador "examesRepetidosEvitados" deve ter aumentado em relação a "antes"

  @negativo @esg-governanca
  Cenário: Indicador não é exposto sem autenticação
    Dado que não estou autenticado
    Quando envio uma requisição GET para "/indicadores/desperdicio"
    Então o status da resposta deve ser 401
    E a resposta deve respeitar o contrato "erro.json"
```

Figura 11 — Exemplo de arquivo .feature no editor — indicador ESG (positivos, regra de negócio e negativo)

01_info_saude.feature

Endpoints públicos de identificação do ambiente/versão e de saúde da aplicação — rastreabilidade do que está implantado (governança). *(2 cenário(s))*

```gherkin
# language: pt
@info @esg-governanca
Funcionalidade: Identificação do ambiente e saúde da aplicação
  Como equipe de operação da HealthTrack
  Quero consultar o ambiente, a versão e a saúde da API
  Para garantir rastreabilidade do que está implantado em cada ambiente (governança)

  @positivo
  Cenário: Consultar ambiente e versão em execução sem autenticação
    Dado que não estou autenticado
    Quando envio uma requisição GET para "/"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "info.json"
    E o campo "aplicacao" deve ser "HealthTrack API"
    E o campo "versao" deve estar preenchido

  @positivo
  Cenário: Verificar que a aplicação está saudável
    Dado que não estou autenticado
    Quando envio uma requisição GET para "/actuator/health"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "health.json"
    E o campo "status" deve ser "UP"
```

02_autenticacao.feature

Registro e login (JWT): caminho feliz, senha incorreta (401), e-mail duplicado (400) e um Esquema do Cenário com 3 exemplos de dados inválidos (400 com a lista de campos com erro). *(4 cenário(s) + 1 esquema(s) de cenário)*

```
# language: pt
@autenticacao @esg-governanca
Funcionalidade: Autenticação e registro de usuários
  Como usuário da HealthTrack (médico, paciente ou administrador)
  Quero me registrar e fazer login
  Para acessar os dados de saúde apenas com credenciais válidas (controle de acesso)

  @positivo
  Cenário: Registrar um novo usuário com dados válidos
    Dado que não estou autenticado
    Quando envio uma requisição POST para "/auth/register" com o corpo:
      """
      {"nome": "Ana Souza", "email": "ana.{execucao}@teste.com", "senha": "senha123",
"tipoUsuario": "MEDICO"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "token.json"
    E o campo "tipo" deve ser "Bearer"

  @positivo
  Cenário: Fazer login com credenciais válidas
    Dado que não estou autenticado
    E envio uma requisição POST para "/auth/register" com o corpo:
      """
      {"nome": "Carlos Lima", "email": "carlos.{execucao}@teste.com", "senha": "senha123",
"tipoUsuario": "PACIENTE"}
      """
    Quando envio uma requisição POST para "/auth/login" com o corpo:
      """
      {"email": "carlos.{execucao}@teste.com", "senha": "senha123"}
      """
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "token.json"
    E o campo "token" deve estar preenchido

  @negativo
  Cenário: Recusar login com senha incorreta
    Dado que não estou autenticado
    E envio uma requisição POST para "/auth/register" com o corpo:
      """
      {"nome": "Beatriz Rocha", "email": "beatriz.{execucao}@teste.com", "senha": "senha123",
"tipoUsuario": "PACIENTE"}
      """
    Quando envio uma requisição POST para "/auth/login" com o corpo:
      """
      {"email": "beatriz.{execucao}@teste.com", "senha": "senhaErrada"}
      """
    Então o status da resposta deve ser 401
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve ser "Credenciais inválidas"

  @negativo
  Cenário: Impedir registro com e-mail já cadastrado
    Dado que não estou autenticado
    E envio uma requisição POST para "/auth/register" com o corpo:
      """
      {"nome": "Diego Prado", "email": "diego.{execucao}@teste.com", "senha": "senha123",
"tipoUsuario": "MEDICO"}
      """
```

```
    Quando envio uma requisição POST para "/auth/register" com o corpo:
      """
      {"nome": "Diego Prado", "email": "diego.{execucao}@teste.com", "senha": "outraSenha",
"tipoUsuario": "MEDICO"}
      """
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve conter "E-mail já cadastrado"

  @negativo
  Esquema do Cenário: Rejeitar registro com dados inválidos (<motivo>)
    Dado que não estou autenticado
    Quando envio uma requisição POST para "/auth/register" com o corpo:
      """
      {"nome": "<nome>", "email": "<email>", "senha": "<senha>", "tipoUsuario": "MEDICO"}
      """
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro-validacao.json"
    E a lista de erros deve apontar o campo "<campo>"

    Exemplos:
      | motivo           | nome      | email                        | senha    | campo |
      | e-mail inválido  | Elisa Paz | email-sem-arroba             | senha123 | email |
      | senha curta      | Elisa Paz | elisa.{execucao}@teste.com   | 123      | senha |
      | nome muito curto | E         | elisa2.{execucao}@teste.com | senha123 | nome  |
```

03_pacientes.feature

CRUD completo de pacientes, validação de CPF/data, integridade (CPF único) e controle de acesso por perfil: PACIENTE não cadastra (403), MÉDICO não remove (403), sem token ou token inválido é bloqueado (401). *(10 cenário(s) + 1 esquema(s) de cenário)*

```
# language: pt
@pacientes
Funcionalidade: Gestão de pacientes
  Como profissional de saúde da HealthTrack
  Quero cadastrar, consultar, atualizar e remover pacientes
  Para manter um prontuário unificado e evitar retrabalho no sistema de saúde

  @positivo @esg-social
  Cenário: Cadastrar um paciente com dados válidos
    Dado que estou autenticado como "MEDICO"
    E que gerei um CPF válido
    Quando envio uma requisição POST para "/pacientes" com o corpo:
      """
      {"nome": "Fernanda Costa", "cpf": "{cpfValido}", "dataNascimento": "1992-07-14",
       "telefone": "(11) 99876-5432", "email": "fernanda.{execucao}@teste.com", "endereco": "Rua
Verde, 10"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "paciente.json"
    E o campo "nome" deve ser "Fernanda Costa"
    E o campo "cpf" deve ser "{cpfValido}"

  @positivo
  Cenário: Listar pacientes de forma paginada
    Dado que estou autenticado como "PACIENTE"
    Quando envio uma requisição GET para "/pacientes?page=0&size=5"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "pagina-pacientes.json"
    E o campo "size" deve ser o número 5
    E a lista "content" não deve estar vazia

  @positivo
```

```
  Cenário: Buscar um paciente pelo ID
    Dado que existe um paciente cadastrado
    E que estou autenticado como "MEDICO"
    Quando envio uma requisição GET para "/pacientes/{pacienteId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "paciente.json"
    E o campo "id" deve ser "{pacienteId}"

  @positivo
  Cenário: Atualizar os dados de contato de um paciente
    Dado que existe um paciente cadastrado
    E que estou autenticado como "ADMIN"
    E envio uma requisição GET para "/pacientes/{pacienteId}"
    E guardo o campo "cpf" da resposta como "cpfAtual"
    Quando envio uma requisição PUT para "/pacientes/{pacienteId}" com o corpo:
      """
      {"nome": "Paciente Atualizado", "cpf": "{cpfAtual}", "dataNascimento": "1990-05-20",
       "telefone": "(11) 90000-1111", "email": "atualizado.{execucao}@teste.com", "endereco":
"Rua Nova, 200"}
      """
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "paciente.json"
    E o campo "nome" deve ser "Paciente Atualizado"
    E o campo "telefone" deve ser "(11) 90000-1111"

  @positivo @esg-governanca
  Cenário: Administrador remove um paciente sem histórico clínico
    Dado que existe um paciente cadastrado
    E que estou autenticado como "ADMIN"
    Quando envio uma requisição DELETE para "/pacientes/{pacienteId}"
    Então o status da resposta deve ser 204
    E o corpo da resposta deve estar vazio
    Quando envio uma requisição GET para "/pacientes/{pacienteId}"
    Então o status da resposta deve ser 404

  @negativo
  Cenário: Buscar um paciente inexistente
    Dado que estou autenticado como "MEDICO"
    Quando envio uma requisição GET para "/pacientes/999999999"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve conter "Paciente não encontrado"

  @negativo
  Cenário: Rejeitar cadastro com CPF inválido e data de nascimento futura
    Dado que estou autenticado como "MEDICO"
    Quando envio uma requisição POST para "/pacientes" com o corpo:
      """
      {"nome": "Gustavo Reis", "cpf": "111.111.111-11", "dataNascimento": "2999-01-01"}
      """
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro-validacao.json"
    E a lista de erros deve apontar o campo "cpf"
    E a lista de erros deve apontar o campo "dataNascimento"

  @negativo @esg-governanca
  Cenário: Impedir cadastro de CPF duplicado (integridade dos dados)
    Dado que estou autenticado como "MEDICO"
    E que gerei um CPF válido
    E envio uma requisição POST para "/pacientes" com o corpo:
      """
      {"nome": "Helena Dias", "cpf": "{cpfValido}", "dataNascimento": "1980-02-02"}
      """
    Quando envio uma requisição POST para "/pacientes" com o corpo:
      """
      {"nome": "Outra Pessoa", "cpf": "{cpfValido}", "dataNascimento": "1981-03-03"}
      """
```

```
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve conter "CPF já cadastrado"

  @negativo @esg-governanca
  Cenário: Paciente não pode cadastrar outros pacientes (controle de acesso)
    Dado que estou autenticado como "PACIENTE"
    E que gerei um CPF válido
    Quando envio uma requisição POST para "/pacientes" com o corpo:
      """
      {"nome": "Igor Mendes", "cpf": "{cpfValido}", "dataNascimento": "1995-09-09"}
      """
    Então o status da resposta deve ser 403
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "error" deve ser "Forbidden"

  @negativo @esg-governanca
  Cenário: Médico não pode remover pacientes (somente ADMIN)
    Dado que existe um paciente cadastrado
    E que estou autenticado como "MEDICO"
    Quando envio uma requisição DELETE para "/pacientes/{pacienteId}"
    Então o status da resposta deve ser 403
    E a resposta deve respeitar o contrato "erro.json"

  @negativo @esg-governanca
  Esquema do Cenário: Bloquear acesso aos dados de saúde sem token válido (<situacao>)
    Dado <condicao>
    Quando envio uma requisição GET para "/pacientes"
    Então o status da resposta deve ser 401
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve ser "Token ausente ou inválido"

    Exemplos:
      | situacao       | condicao                          |
      | sem token      | que não estou autenticado         |
      | token inválido | que estou usando um token inválido |
```

04_prontuarios.feature

Prontuário eletrônico unificado: registrar atendimento, consultar histórico paginado (todos os itens do paciente), campo obrigatório ausente (400) e paciente inexistente (404). *(4 cenário(s))*

```
# language: pt
@prontuarios @esg-social
Funcionalidade: Prontuário eletrônico unificado
  Como médico da HealthTrack
  Quero registrar e consultar atendimentos de um paciente
  Para que o histórico clínico fique disponível para qualquer unidade de saúde

  Contexto:
    Dado que existe um paciente cadastrado
    E que existe um médico cadastrado

  @positivo
  Cenário: Registrar um atendimento no prontuário do paciente
    Dado que estou autenticado como "MEDICO"
    Quando envio uma requisição POST para "/prontuarios" com o corpo:
      """
      {"pacienteId": {pacienteId}, "medicoId": {medicoId}, "dataAtendimento": "2026-09-01",
       "diagnostico": "Rinite alérgica", "prescricao": "Loratadina 10mg", "observacoes": "Retorno
em 30 dias"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "prontuario.json"
```

```
    E o campo "pacienteId" deve ser "{pacienteId}"
    E o campo "diagnostico" deve ser "Rinite alérgica"

  @positivo
  Cenário: Consultar o histórico clínico do paciente
    Dado que estou autenticado como "MEDICO"
    E envio uma requisição POST para "/prontuarios" com o corpo:
      """
      {"pacienteId": {pacienteId}, "medicoId": {medicoId}, "dataAtendimento": "2026-08-15",
       "diagnostico": "Check-up anual"}
      """
    Quando envio uma requisição GET para "/prontuarios/{pacienteId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "pagina-prontuarios.json"
    E a lista "content" não deve estar vazia
    E todos os itens de "content" devem ter o campo "pacienteId" igual a "{pacienteId}"

  @negativo
  Cenário: Rejeitar atendimento sem diagnóstico
    Dado que estou autenticado como "MEDICO"
    Quando envio uma requisição POST para "/prontuarios" com o corpo:
      """
      {"pacienteId": {pacienteId}, "medicoId": {medicoId}, "dataAtendimento": "2026-09-01"}
      """
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro-validacao.json"
    E a lista de erros deve apontar o campo "diagnostico"

  @negativo
  Cenário: Consultar histórico de paciente inexistente
    Dado que estou autenticado como "MEDICO"
    Quando envio uma requisição GET para "/prontuarios/999999999"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
```

05_exames.feature

Histórico de exames: registrar, consultar histórico, tipo inexistente (400 com os valores aceitos) e paciente inexistente (404). *(4 cenário(s))*

```
# language: pt
@exames @esg-social
Funcionalidade: Histórico de exames
  Como médico da HealthTrack
  Quero registrar e consultar os exames de um paciente
  Para evitar a repetição desnecessária de exames (redução de desperdício)

  Contexto:
    Dado que existe um paciente cadastrado
    E que estou autenticado como "MEDICO"

  @positivo
  Cenário: Registrar um exame para o paciente
    Quando envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "SANGUE", "descricao": "Glicemia em jejum",
       "dataRealizacao": "2026-09-10", "resultado": "92 mg/dL", "laboratorio": "Laboratório
Central"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "exame.json"
    E o campo "tipoExame" deve ser "SANGUE"
    E o campo "pacienteId" deve ser "{pacienteId}"
```

```
  @positivo
  Cenário: Consultar o histórico de exames do paciente
    Dado envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "IMAGEM", "descricao": "Raio-X de tórax",
"dataRealizacao": "2026-07-01"}
      """
    Quando envio uma requisição GET para "/exames/{pacienteId}/historico"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "pagina-exames.json"
    E a lista "content" não deve estar vazia
    E todos os itens de "content" devem ter o campo "pacienteId" igual a "{pacienteId}"

  @negativo
  Cenário: Rejeitar exame com tipo inexistente
    Quando envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "DNA", "descricao": "Teste", "dataRealizacao":
"2026-09-10"}
      """
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve conter "Valores aceitos"

  @negativo
  Cenário: Rejeitar exame para paciente inexistente
    Quando envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": 999999999, "tipoExame": "URINA", "descricao": "Urina tipo 1",
"dataRealizacao": "2026-09-10"}
      """
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
    E o campo "message" deve conter "Paciente não encontrado"
```

06_indicadores_esg.feature

Indicador ESG de exames repetidos evitados: contrato, regra de negócio (dois exames iguais para o mesmo paciente aumentam o indicador) e proteção do endpoint (401). *(3 cenário(s))*

```
# language: pt
@indicadores @esg-social
Funcionalidade: Indicador ESG de desperdício evitado
  Como gestor de saúde pública
  Quero acompanhar quantos exames repetidos foram evitados e a economia gerada
  Para medir o impacto social da plataforma

  @positivo
  Cenário: Consultar o indicador de exames repetidos evitados
    Dado que estou autenticado como "ADMIN"
    Quando envio uma requisição GET para "/indicadores/desperdicio"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "indicador-desperdicio.json"
    E o campo "impactoESG" deve conter "Social"

  @positivo
  Cenário: Exame repetido entra no cálculo do indicador
    Dado que existe um paciente cadastrado
    E que estou autenticado como "MEDICO"
    E envio uma requisição GET para "/indicadores/desperdicio"
    E guardo o campo "examesRepetidosEvitados" da resposta como "antes"
    E envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "CARDIOLOGICO", "descricao": "Eletrocardiograma",
```

```
"dataRealizacao": "2026-09-01"}
      """
    E envio uma requisição POST para "/exames" com o corpo:
      """
      {"pacienteId": {pacienteId}, "tipoExame": "CARDIOLOGICO", "descricao": "Eletrocardiograma",
"dataRealizacao": "2026-09-20"}
      """
    Quando envio uma requisição GET para "/indicadores/desperdicio"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "indicador-desperdicio.json"
    E o indicador "examesRepetidosEvitados" deve ter aumentado em relação a "antes"

  @negativo @esg-governanca
  Cenário: Indicador não é exposto sem autenticação
    Dado que não estou autenticado
    Quando envio uma requisição GET para "/indicadores/desperdicio"
    Então o status da resposta deve ser 401
    E a resposta deve respeitar o contrato "erro.json"
```

5. Testes de API: status code, corpo JSON e contrato

Todos os 14 endpoints são exercitados. Para cada chamada, a suíte valida (1) o **status code**, (2) o **corpo JSON** (valores de campos, listas e mensagens) e (3) o **contrato** com JSON Schema — inclusive nas respostas de erro (`erro.json` / `erro-validacao.json`).

Método	Endpoint	Feature	Sucesso	Contrato	Cenários negativos
GET	/	01	200	info.json	—
GET	/actuator/health	01	200	health.json	—
POST	/auth/register	02	201	token.json	400 e-mail duplicado / e-mail inválido / senha curta / nome curto
POST	/auth/login	02	200	token.json	401 senha incorreta
GET	/pacientes	03	200	pagina-pacientes.json	401 sem token / token inválido
GET	/pacientes/{id}	03	200	paciente.json	404 inexistente
POST	/pacientes	03	201	paciente.json	400 CPF inválido / duplicado; 403 perfil PACIENTE
PUT	/pacientes/{id}	03	200	paciente.json	—
DELETE	/pacientes/{id}	03	204 (corpo vazio)	–	403 perfil MÉDICO
POST	/prontuarios	04	201	prontuario.json	400 sem diagnóstico
GET	/prontuarios/{pacienteId}	04	200	pagina-prontuarios.json	404 inexistente
POST	/exames	05	201	exame.json	400 tipo inexistente; 404 paciente inexistente
GET	/exames/{pacienteId}/historico	05	200	pagina-exames.json	—

GET	/indicadores/desperdicio	06	200	indicador-desperdicio.json	401 sem token

Exemplo real de comunicação com a API

Requisições e respostas reais da API em execução, mostrando os mesmos comportamentos validados pela suíte (201 no cadastro, 403 por perfil sem permissão e o corpo do indicador ESG):

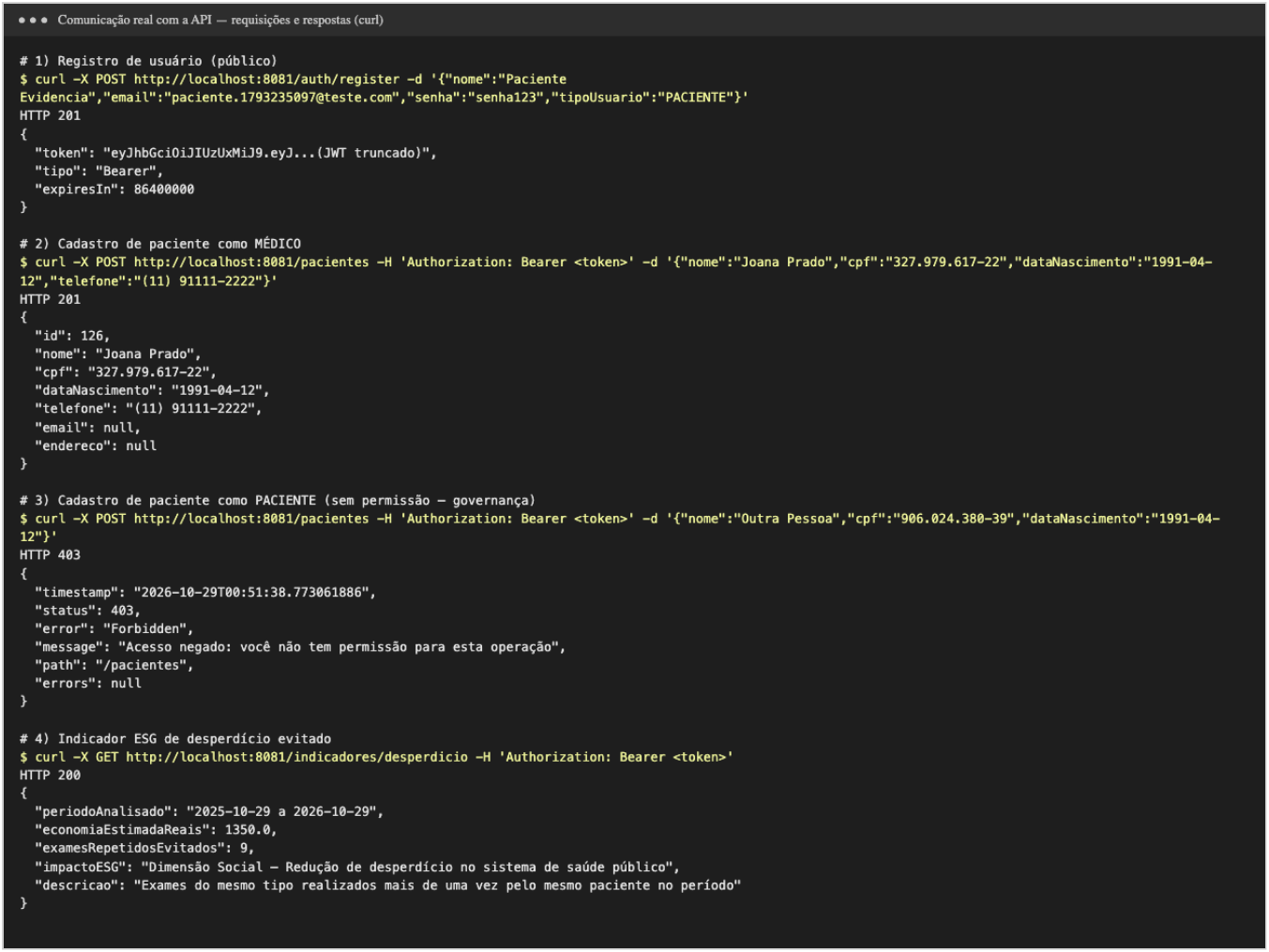


Figura 12 — Requisições e respostas reais (curl) — status code e corpo JSON

6. Como executar

Forma	Comando (na raiz do projeto)
Tudo em containers (só precisa de Docker)	`./testes-bdd/run-tests.sh --docker`
API em container + testes com Maven local	`./testes-bdd/run-tests.sh` (porta ocupada: `APP_PORT=8081 ./testes-bdd/run-tests.sh`)
Contra uma API já em execução	`cd testes-bdd && mvn clean test -DbaseUrl=https://url-da-api`
Somente um grupo de cenários (tag)	`mvn clean test -DbaseUrl=... -Dcucumber.filter.tags="@esg-governanca"`
CI (GitHub Actions)	Automático em push / Pull Request, ou manual em *Actions → Testes BDD e de API → Run workflow*

Relatórios: `testes-bdd/target/cucumber-reports/relatorio.html` (visual), `relatorio.json` e `target/surefire-reports/` (JUnit).

7. Execução detalhada e evidências

7.1 Passo 1 — subir o ambiente

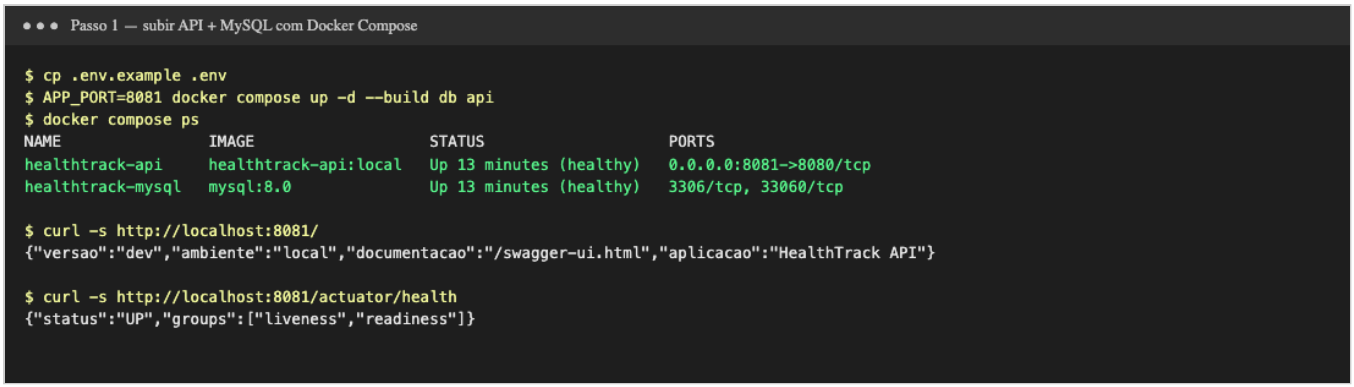


Figura 13 — API e MySQL em execução via Docker Compose (containers healthy) e endpoints públicos respondendo

7.2 Passo 2 — executar a suíte

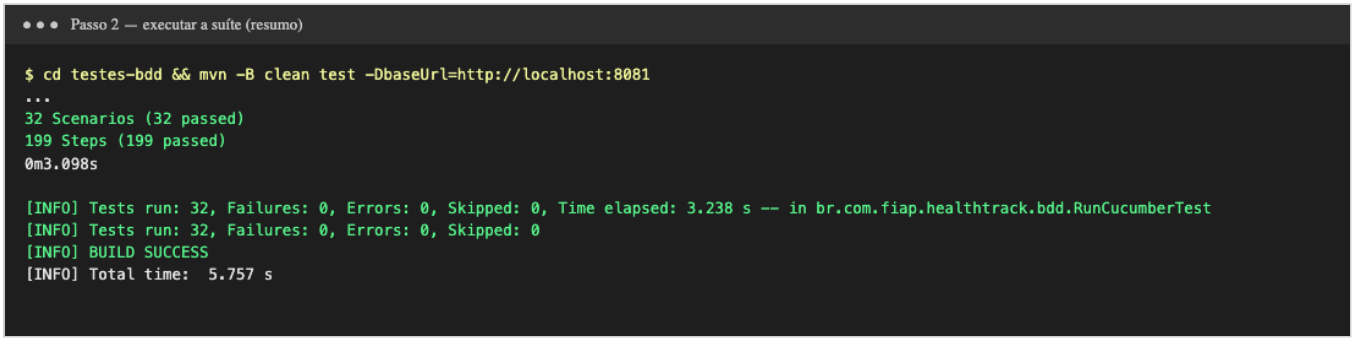


Figura 14 — Resumo da execução: 32 cenários e 199 passos aprovados, BUILD SUCCESS



Figura 15 — Saída do Cucumber durante a execução — cenários de pacientes, passo a passo

7.3 Resultado por cenário

Tabela extraída do relatório JSON gerado pelo Cucumber na execução: cada cenário, suas tags, quantidade de passos, status e tempo.

Resultado por cenário — extraído de target/cucumber-reports/relatorio.json

#	Feature	Cenário	Tags	Passos	Status	Tempo
1	01_info_saude.feature	Consultar ambiente e versão em execução sem autenticação	@info @esg-governanca @positivo	6	✔ passed	856 ms
2	01_info_saude.feature	Verificar que a aplicação está saudável	@info @esg-governanca @positivo	5	✔ passed	32 ms
3	02_autenticacao.feature	Registrar um novo usuário com dados válidos	@esg-governanca @positivo	5	✔ passed	130 ms
4	02_autenticacao.feature	Fazer login com credenciais válidas	@esg-governanca @positivo	6	✔ passed	182 ms
5	02_autenticacao.feature	Recusar login com senha incorreta	@esg-governanca @negativo	6	✔ passed	169 ms
6	02_autenticacao.feature	Impedir registro com e-mail já cadastrado	@esg-governanca @negativo	6	✔ passed	108 ms
7	02_autenticacao.feature	Rejeitar registro com dados inválidos (e-mail inválido)	@esg-governanca @negativo	5	✔ passed	33 ms
8	02_autenticacao.feature	Rejeitar registro com dados inválidos (senha curta)	@esg-governanca @negativo	5	✔ passed	17 ms
9	02_autenticacao.feature	Rejeitar registro com dados inválidos (nome muito curto)	@esg-governanca @negativo	5	✔ passed	15 ms
10	03_pacientes.feature	Cadastrar um paciente com dados válidos	@positivo @esg-social	7	✔ passed	142 ms
11	03_pacientes.feature	Listar pacientes de forma paginada	@positivo	6	✔ passed	130 ms
12	03_pacientes.feature	Buscar um paciente pelo ID	@positivo	6	✔ passed	154 ms
13	03_pacientes.feature	Atualizar os dados de contato de um paciente	@positivo	9	✔ passed	81 ms
14	03_pacientes.feature	Administrador remove um paciente sem histórico clínico	@positivo @esg-governanca	7	✔ passed	49 ms
15	03_pacientes.feature	Buscar um paciente inexistente	@negativo	5	✔ passed	16 ms
16	03_pacientes.feature	Rejeitar cadastro com CPF inválido e data de nascimento futura	@negativo	6	✔ passed	28 ms
17	03_pacientes.feature	Impedir cadastro de CPF duplicado (integridade dos dados)	@negativo @esg-governanca	7	✔ passed	35 ms
18	03_pacientes.feature	Paciente não pode cadastrar outros pacientes (controle de acesso)	@negativo @esg-governanca	6	✔ passed	19 ms
19	03_pacientes.feature	Médico não pode remover pacientes (somente ADMIN)	@negativo @esg-governanca	5	✔ passed	31 ms
20	03_pacientes.feature	Bloquear acesso aos dados de saúde sem token válido (sem token)	@negativo @esg-governanca	5	✔ passed	13 ms
21	03_pacientes.feature	Bloquear acesso aos dados de saúde sem token válido (token inválido)	@negativo @esg-governanca	5	✔ passed	13 ms
22	04_prontuarios.feature	Registrar um atendimento no prontuário do paciente	@esg-social @positivo	8	✔ passed	121 ms
23	04_prontuarios.feature	Consultar o histórico clínico do paciente	@esg-social @positivo	9	✔ passed	113 ms
24	04_prontuarios.feature	Rejeitar atendimento sem diagnóstico	@esg-social @negativo	7	✔ passed	93 ms
25	04_prontuarios.feature	Consultar histórico de paciente inexistente	@esg-social @negativo	6	✔ passed	72 ms
26	05_exames.feature	Registrar um exame para o paciente	@esg-social @positivo	7	✔ passed	43 ms
27	05_exames.feature	Consultar o histórico de exames do paciente	@esg-social @positivo	8	✔ passed	52 ms
28	05_exames.feature	Rejeitar exame com tipo inexistente	@esg-social @negativo	6	✔ passed	30 ms
29	05_exames.feature	Rejeitar exame para paciente inexistente	@esg-social @negativo	6	✔ passed	33 ms
30	06_indicadores_esg.feature	Consultar o indicador de exames repetidos evitados	@indicadores @esg-social @positivo	5	✔ passed	22 ms
31	06_indicadores_esg.feature	Exame repetido entra no cálculo do indicador	@indicadores @esg-social @positivo	10	✔ passed	68 ms
32	06_indicadores_esg.feature	Indicador não é exposto sem autenticação	@indicadores @esg-social @negativo @esg-governanca	4	✔ passed	10 ms
	TOTAL: 32 cenários — 32 aprovados			**199**	**32/32**	**2909 ms**

Figura 16 — Resultado dos 32 cenários — 32/32 aprovados, 199 passos

7.4 Relatório HTML do Cucumber

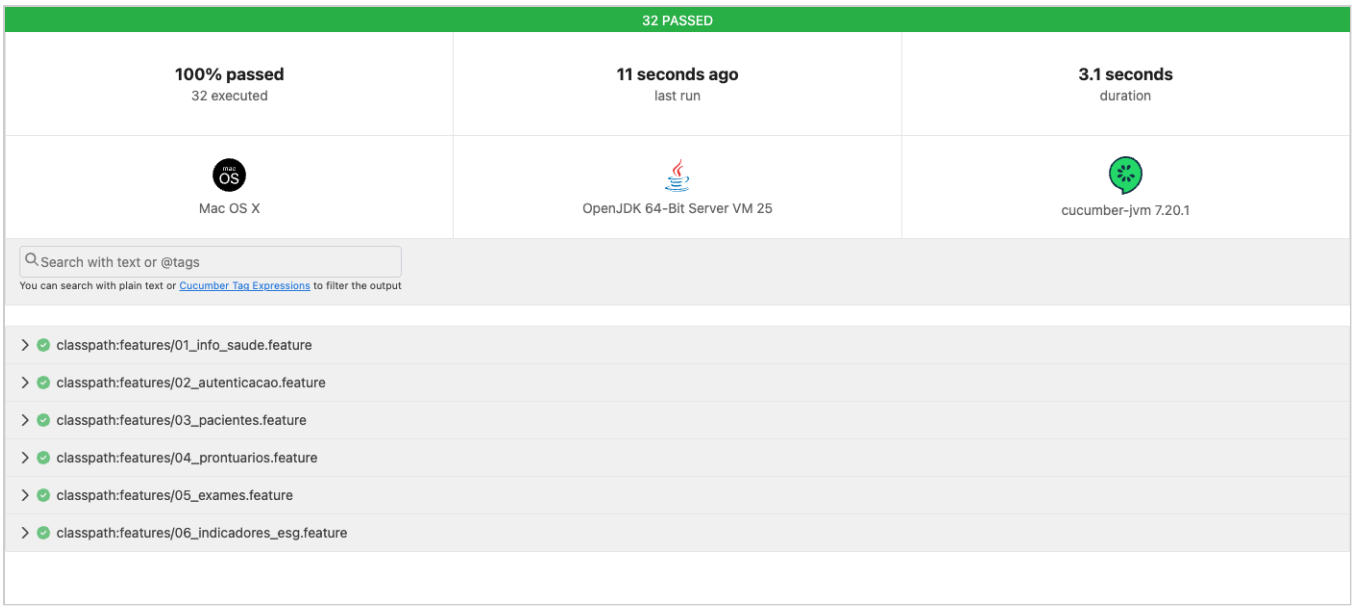


Figura 17 — Relatório HTML do Cucumber: 100% aprovado nas 6 funcionalidades

7.5 Execução 100% em containers

Com um único comando, o Docker Compose sobe MySQL e API, espera o healthcheck e executa o container de testes; o comando retorna o código de saída dos testes.

```
● ● ●  Execução completa em containers — ./testes-bdd/run-tests.sh --docker

$ APP_PORT=8081 ./testes-bdd/run-tests.sh --docker
>> Executando API + testes BDD em containers
 Container healthtrack-mysql  Created
 Container healthtrack-api  Recreated
 Container healthtrack-testes-bdd  Recreated
...
healthtrack-api         | 2026-10-29T00:37:05.607Z  INFO 1 --- [healthtrack] [           main] b.c.f.h.HealthtrackApplication           : Started
HealthtrackApplication in 4.524 seconds (process runni
...
healthtrack-testes-bdd  | Cenário: Consultar ambiente e versão em execução sem autenticação # features/01_info_saude.feature:9
healthtrack-testes-bdd  | Cenário: Verificar que a aplicação está saudável          # features/01_info_saude.feature:18
healthtrack-testes-bdd  | Cenário: Registrar um novo usuário com dados válidos                # features/02_autenticacao.feature:9
healthtrack-testes-bdd  | Cenário: Fazer login com credenciais válidas                      # features/02_autenticacao.feature:20
healthtrack-testes-bdd  | Cenário: Recusar login com senha incorreta                        # features/02_autenticacao.feature:35
healthtrack-testes-bdd  | Cenário: Impedir registro com e-mail já cadastrado                   # features/02_autenticacao.feature:50
healthtrack-testes-bdd  | ... (32 cenários executados)
healthtrack-testes-bdd  | 32 Scenarios (32 passed)
healthtrack-testes-bdd  | 199 Steps (199 passed)
healthtrack-testes-bdd  | 0m3.835s
healthtrack-testes-bdd  | [INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0
healthtrack-testes-bdd  | [INFO] BUILD SUCCESS

healthtrack-testes-bdd exited with code 0
>> Relatório: testes-bdd/target/cucumber-reports/relatorio.html
```

Figura 18 — ./testes-bdd/run-tests.sh --docker — ambiente e testes em containers, 32/32 aprovados

7.6 Execução filtrada por tag (pilar ESG)

Execução filtrada por tag — só cenários de Governança (ESG)

```
$ mvn clean test -DbaseUrl=http://localhost:8081 -Dcucumber.filter.tags="@esg-governanca"
...
Cenário: Consultar ambiente e versão em execução sem autenticação
Cenário: Verificar que a aplicação está saudável
Cenário: Registrar um novo usuário com dados válidos
Cenário: Fazer login com credenciais válidas
Cenário: Recusar login com senha incorreta
Cenário: Impedir registro com e-mail já cadastrado
Esquema do Cenário: Rejeitar registro com dados inválidos (e-mail inválido)
Esquema do Cenário: Rejeitar registro com dados inválidos (senha curta)
Esquema do Cenário: Rejeitar registro com dados inválidos (nome muito curto)
Cenário: Administrador remove um paciente sem histórico clínico
Cenário: Impedir cadastro de CPF duplicado (integridade dos dados)
Cenário: Paciente não pode cadastrar outros pacientes (controle de acesso)
Cenário: Médico não pode remover pacientes (somente ADMIN)
Esquema do Cenário: Bloquear acesso aos dados de saúde sem token válido (sem token)
Esquema do Cenário: Bloquear acesso aos dados de saúde sem token válido (token inválido)
Cenário: Indicador não é exposto sem autenticação

16 Scenarios (16 passed)
88 Steps (88 passed)
[WARNING] Tests run: 32, Failures: 0, Errors: 0, Skipped: 16
[INFO] BUILD SUCCESS
```

Figura 19 — Somente os 16 cenários de Governança (@esg-governanca) — todos aprovados

7.7 Prova de que os testes detectam falhas

Para validar a própria suíte, um cenário temporário exigiu o contrato errado (`token.json`) na resposta de `GET /`. O teste **falhou**, indicando exatamente os campos ausentes; o arquivo foi removido em seguida.

Prova: o teste FALHA quando a resposta não respeita o contrato

```
# Cenário TEMPORÁRIO, criado só para provar que a validação de contrato funciona (removido em seguida):
$ cat features/99_prova_falha.feature
# language: pt
@prova-falha
Funcionalidade: Prova de que o contrato detecta quebras
  Cenário: Resposta de GET / validada contra o contrato errado
    Dado que não estou autenticado
    Quando envio uma requisição GET para "/"
    Então a resposta deve respeitar o contrato "token.json"

$ mvn clean test -DbaseUrl=http://localhost:8081 -Dcucumber.filter.tags="@prova-falha"
...
Cenário: Resposta de GET / validada contra o contrato errado
  Dado que não estou autenticado
  Quando envio uma requisição GET para "/"
  Então a resposta deve respeitar o contrato "token.json"
      java.lang.AssertionError: 1 expectation failed.
Response body doesn't match expectation.
error: object has missing required properties (["expiresIn","tipo","token"])
    level: "error"
    schema: {"loadingURI":"file:/Users/nsantana/Documents/FIAP/cidades-esg-inteligentes/testes-bdd/target/test-
classes/schemas/token.json#","pointer":""}
    instance: {"pointer":""}
    domain: "validation"
    keyword: "required"
    required: ["expiresIn","tipo","token"]

1 Scenarios (1 failed)
3 Steps (1 failed, 2 passed)
[INFO] BUILD FAILURE
```

Figura 20 — Contrato quebrado → teste falha com a mensagem do JSON Schema (required: expiresIn, tipo, token)

7.8 Execução no pipeline de CI (GitHub Actions)

O workflow da seção 3.9 foi executado no GitHub Actions a cada push na branch `feature/testes-bdd`: cria um .env com segredos aleatórios, sobe API + MySQL, aguarda a saúde da API, roda a suíte e publica o relatório do Cucumber como artefato.

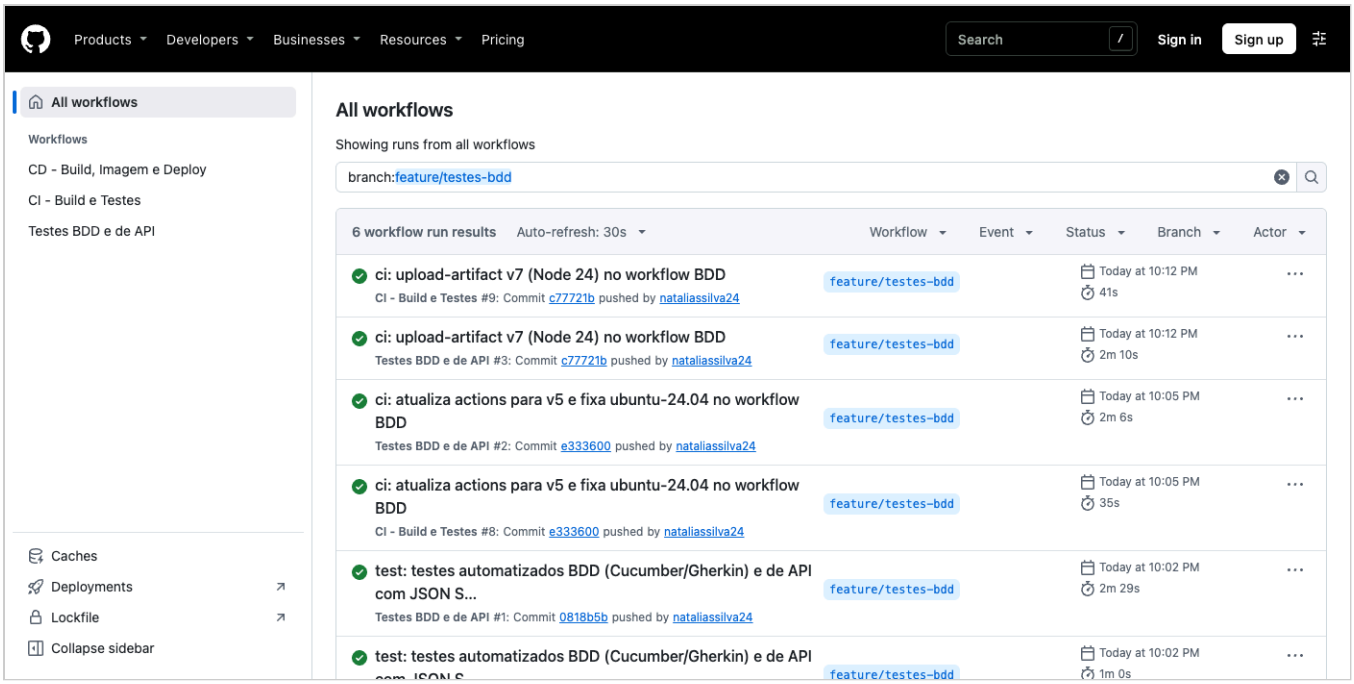


Figura 21 — GitHub Actions — execuções da branch feature/testes-bdd: workflow de testes BDD e CI do projeto, todas com sucesso

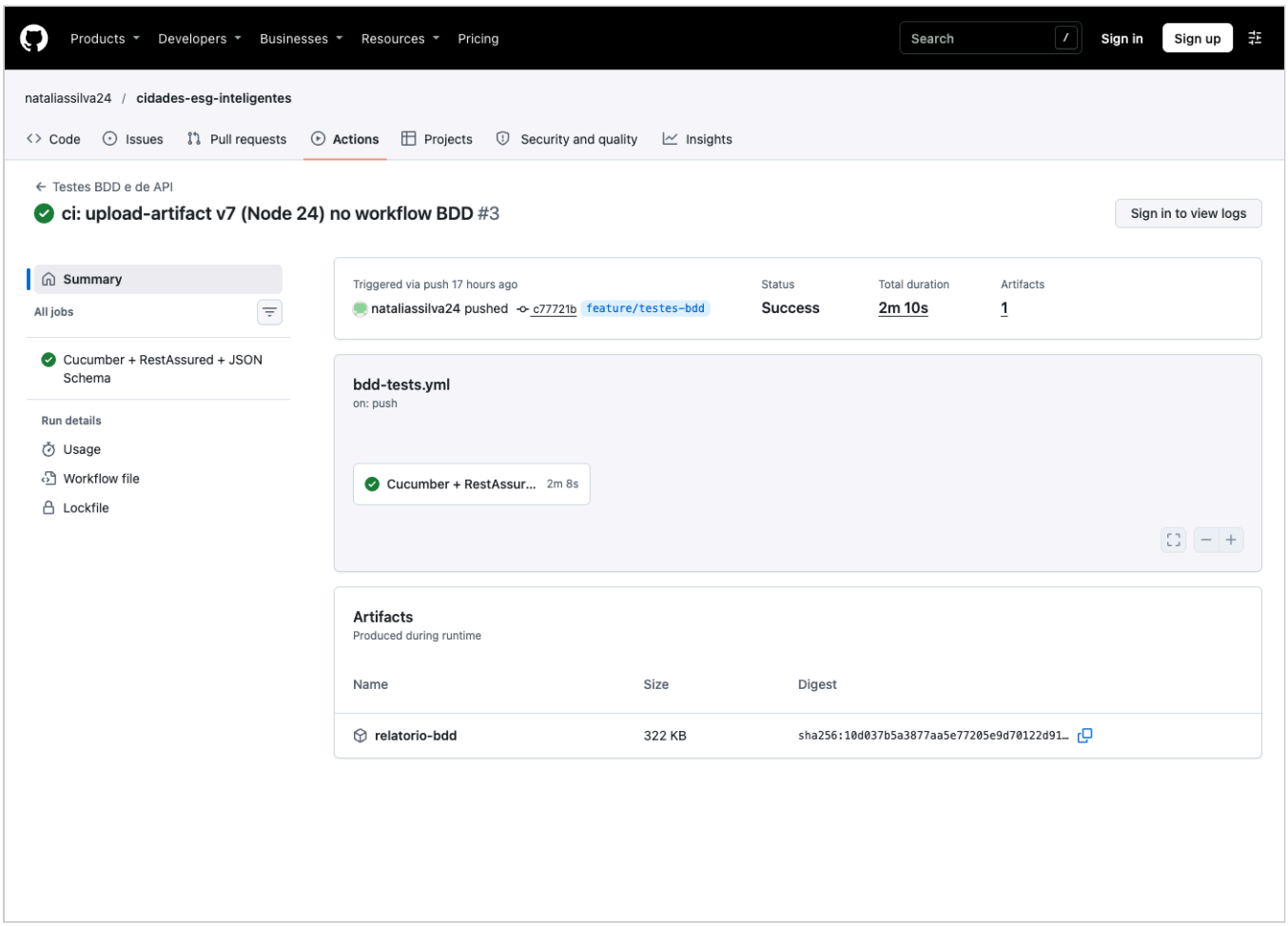


Figura 22 — Execução do workflow "Testes BDD e de API": status Success, duração e artefato relatorio-bdd publicado

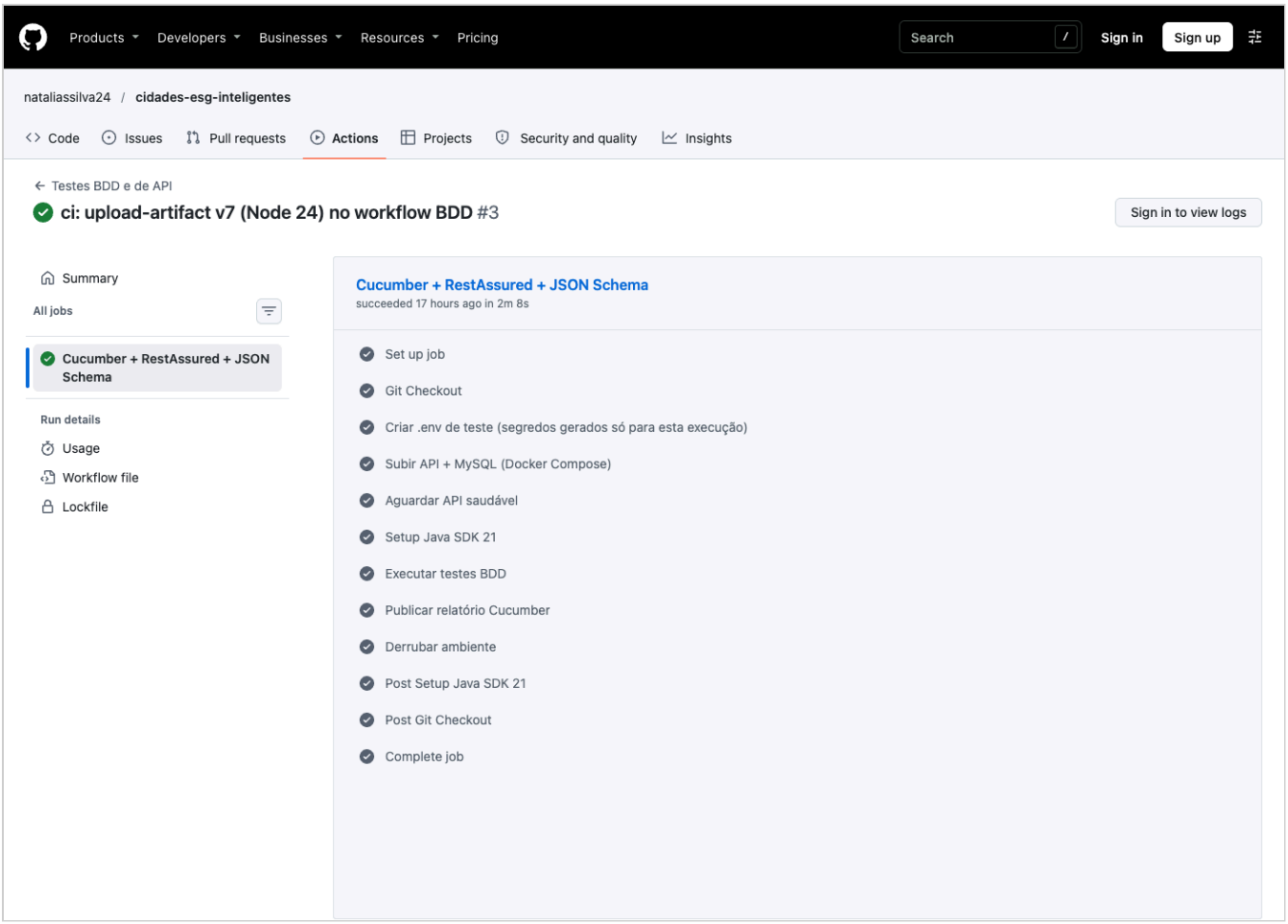


Figura 23 — Etapas do job: cria .env de teste, sobe API + MySQL, aguarda saúde, executa os testes, publica o relatório e derruba o ambiente

Log real do GitHub Actions — passo "Executar testes BDD"

```
# GitHub Actions — run #3 (feature/testes-bdd) — job "Cucumber + RestAssured + JSON Schema"
$ docker compose up -d --build db api
  Container healthtrack-mysql  Started
  Container healthtrack-mysql  Healthy
  Container healthtrack-api  Started
  {"status":"UP","groups":["liveness","readiness"]}
$ mvn -B clean test -DbaseUrl=http://localhost:8080
...
Cenário: Consultar ambiente e versão em execução sem autenticação
Cenário: Verificar que a aplicação está saudável
Cenário: Registrar um novo usuário com dados válidos
Cenário: Fazer login com credenciais válidas
Cenário: Recusar login com senha incorreta
Cenário: Impedir registro com e-mail já cadastrado
... (32 cenários executados)

32 Scenarios (32 passed)
199 Steps (199 passed)
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  17.521 s
Artifact relatorio-bdd successfully finalized. Artifact ID 12865571011
Artifact relatorio-bdd has been successfully uploaded! Final size is 329631 bytes. Artifact ID is 12865571011
Artifact download URL: https://github.com/nataliassilva24/cidades-esg-inteligentes/actions/runs/40798947409/artifacts/12865571011
```

Figura 24 — Log real do passo "Executar testes BDD" no GitHub Actions: 32 cenários e 199 passos aprovados

8. Conclusão

A suíte cobre os 14 endpoints da HealthTrack API com 32 cenários BDD (199 passos), incluindo caminhos felizes e de falha, com validação de status code, corpo JSON e contrato via JSON Schema. A execução é reproduzível com um único comando — localmente ou 100% em Docker — e automatizada no GitHub Actions. Além da qualidade técnica, os testes asseguram requisitos ligados aos pilares ESG: o cálculo correto do indicador de desperdício evitado (Social) e o controle de acesso e a integridade dos dados de saúde (Governança).